الـ Dependency Injection في المشروع

مرجع لأنواع التسجيل في get_it وليه اخترنا كل واحدة

الحزمة: get_it ^9.2.1 · ملف التركيب: lib/core/di/injection.dart

---

المقارنة السريعة

النوع	كام نسخة؟	بتتعمل امتى؟	مين بيقفلها؟
`registerFactory`	جديدة كل نداء	وقت النداء	انت / اللي بياخدها
`registerLazySingleton`	واحدة	**أول** نداء	`get_it` عند `reset()`
`registerSingleton`	واحدة	**فوراً** وقت التسجيل	`get_it` عند `reset()`
`registerFactoryParam`	جديدة كل نداء	وقت النداء بمعطيات	انت
`registerCachedFactory`	واحدة طول ما حد ماسكها	وقت النداء	الـ GC
`*Async`	حسب النوع	مع `await`	حسب النوع

١) registerFactory — نسخة جديدة كل مرة

```
sl.registerFactory(() => LoginUseCase(sl()));

final a = sl<LoginUseCase>();
final b = sl<LoginUseCase>();
identical(a, b);   // false
```

استخدمها لما: الحاجة ليها حالة مؤقتة، أو حد تاني بيمتلكها ويقفلها.

في المشروع: كل الـ Cubits والـ UseCases.

> ⚠️ **مهم:** `get_it` **مبيتابعش** نسخ الـ factory ولا بيقفلها. انت مسؤول عنها.

قاعدة: أي Cubit مع BlocProvider لازم يكون factory

```
BlocProvider(create: (_) => sl<MyCubit>())
```

كلمة `create` معناها إن البروفايدر **بيمتلك** الـ Cubit، فبينادي `close()` عليه لما الويدجت يتشال. لو الـ Cubit مسجّل `lazySingleton`، `get_it` هيفضل ماسك النسخة المقفولة ويوزّعها — وأول `emit` بعد كده يرمي `StateError`.

```
sl.registerFactory(() => LocaleCubit(sl()));        // ✅ صح
sl.registerLazySingleton(() => LocaleCubit(sl())); // ❌ لغم
```

البديل لو محتاج singleton فعلاً: `BlocProvider.value(value: sl<MyCubit>())` — دي بتقول للبروفايدر «انت مش المالك، متقفلهاش».

٢) registerLazySingleton — واحدة، بتتأجل

```
sl.registerLazySingleton(() => ApiClient(tokenStore: sl(), ...));

// وقت التسجيل: مفيش حاجة اتعملت
final a = sl<ApiClient>();   // ← دلوقتي اتعملت
final b = sl<ApiClient>();   // ← نفس النسخة
```

دي الافتراضي اللي تبدأ بيه. بتوفّر وقت بدء التطبيق، وبتخلي ترتيب السطور أقل حساسية لأن الدالة بتتنفّذ لما حد يطلب — وساعتها كل حاجة تكون متسجّلة.

في المشروع: `ApiClient` , `NetworkInfo` , `Connectivity` , `SecureStore` , `AuthRemoteDataSource` , `AuthRepository` .

٣) registerSingleton — واحدة، دلوقتي حالاً

```
registerLazySingleton(() => X())   // دالة ← استناها
registerSingleton(X())             // كائن ← معمول خلاص
```

استخدمها لما: الكائن محتاج تحضير `async` قبله، أو لازم يكون موجود قبل أي حاجة تانية.

في المشروع — تلات حالات، كل واحدة ليها سبب:

```
sl.registerSingleton(logger);                              // ١
sl.registerSingleton(await SharedPreferences.getInstance()); // ٢
sl.registerSingleton(tokenStore);                          // ٣
```

- **AppLogger** — أول سطر في `init()` عشان يقدر يسجّل فشل التهيئة نفسها
- **SharedPreferences** — `getInstance()` دالة async، و `registerLazySingleton` بياخد دالة عادية. وكمان `LocaleCubit` بيقراها بشكل **متزامن** في الكونستركتور
- **TokenStore** — الحالة الأهم، شوف تحت

الحالة الخاصة: TokenStore

```
final tokenStore = TokenStore(sl());   // ١) اعمله
await tokenStore.restore();            // ٢) املاه من الـ keychain
sl.registerSingleton(tokenStore);      // ٣) وبعدين سجّله
```

ليه تلات خطوات؟ لأن `restore()` دالة `async`، ولازم تخلص **قبل** ما الـ router يسأل `isLoggedIn`:

```
التطبيق يفتح
← الراوتر يسأل: isLoggedIn؟
← TokenStore لسه ما قراش من الـ keychain
← يقول «لأ»
← المستخدم يترمي على اللوجين وهو معاه جلسة صالحة
```

مينفعش `registerLazySingleton(() async => ...)` — الدالة لازم تكون sync.

٤) registerFactoryParam — نسخة بمعطيات وقت التشغيل

```
sl.registerFactoryParam<ProductDetailsCubit, String, void>(
  (productId, _) => ProductDetailsCubit(sl(), productId: productId),
);

BlocProvider(
  create: (_) => sl<ProductDetailsCubit>(param1: '12345'),
  child: const ProductDetailsScreen(),
)
```

`<T, P1, P2>` — لو محتاج معطى واحد بس، حط `void` مكان التاني.

من غيرها كنت هتضطر تعمل الـ Cubit من غير الـ id وبعدين تبعته بميثود منفصلة — وده بيسيب فجوة إن الـ Cubit موجود في حالة ناقصة.

مش مستخدمة في المشروع دلوقتي، بس هتحتاجها في أول شاشة تفاصيل.

٥) registerCachedFactory — الوسط

```
sl.registerCachedFactory(() => HeavyImageCache());
```

بيدي نفس النسخة طول ما فيه حد ماسكها. أول ما الـ GC يلمّها، النداء الجاي بيعمل واحدة جديدة. نادرة — مفيدة لحاجات تقيلة عايز تشاركها مؤقتاً من غير ما تفضل في الذاكرة للأبد.

٦) الأنواع الـ Async

```
sl.registerSingletonAsync<TokenStore>(() async {
  final store = TokenStore(sl());
  await store.restore();
  return store;
});

// وفي main
await sl.allReady();
runApp(const App());
```

ولما حاجة async تعتمد على حاجة async تانية:

```
sl.registerSingletonWithDependencies<ApiClient>(
  () => ApiClient(tokenStore: sl(), ...),
  dependsOn: [TokenStore],
);
```

ليه المشروع مش مستخدمها؟ الطريقة اليدوية أوضح في ملف صغير — الترتيب مقروء من فوق لتحت ومفيش سحر مخفي. انقل لـ async لما يبقى عندك 5 أو 6 حاجات async متشابكة، وساعتها `dependsOn` هتتولى الترتيب بدلك.

حاجات كمان

التسجيل بواجهة مجرّدة

```
sl.registerLazySingleton<AuthRepository>(() => AuthRepositoryImpl(sl(), sl()));
```

لاحظ إن النوع بين الأقواس هو `AuthRepository` — **العقد** — مش `AuthRepositoryImpl` اللي هو التنفيذ.

النوع بين `<>` هو اللي `get_it` بيدوّر بيه. فالـ Cubits شايفة `AuthRepository` بس ومتعرفش `AuthRepositoryImpl` أصلاً — ودي نص فكرة الـ Clean Architecture.

إزاي sl() بتعرف تجيب إيه؟

```
ApiClient(
  tokenStore:  sl(),   → TokenStore
  networkInfo: sl(),   → NetworkInfo
  logger:      sl(),   → AppLogger
)
```

من **نوع الباراميتر** مش من اسمه. لو محتاج نوع مختلف عن المتوقع، اكتبه صراحة:

```
AuthRemoteDataSource(sl<ApiClient>().dio)   // عايزين الـ dio مش الـ ApiClient
```

نسخ بأسماء

```
sl.registerLazySingleton<Dio>(() => mainDio, instanceName: 'prism');
sl.registerLazySingleton<Dio>(() => uploadDio, instanceName: 'files');

sl<Dio>(instanceName: 'prism');
```

فحص وتصفير

```
sl.isRegistered<ApiClient>();
await sl.reset();               // مفيد في الاختبارات
```

غلطتين شائعتين

١) sl() جوه تسجيل eager قبل ما التبعية تتسجّل

```
sl.registerSingleton(ApiClient(tokenStore: sl()));           // ❌ بيتنفّذ فوراً
sl.registerLazySingleton(() => ApiClient(tokenStore: sl())); // ✅ بيتأجل
```

٢) Cubit كـ lazySingleton مع BlocProvider — شوف القاعدة فوق.

القرار في ٣ أسئلة

محتاج `await` قبل ما تسجّله؟
أيوة ← اعمله بإيدك + `registerSingleton`
لأ ↓

محتاج نسخة جديدة كل مرة؟
أيوة ← `registerFactory` (+ `Param` لو محتاج معطيات)
لأ ← `registerLazySingleton` ← **الافتراضي**

الوضع الحالي في المشروع

الحاجة	التسجيل	ليه
`AppLogger`	Singleton	أول حاجة، عشان يسجّل فشل التهيئة
`SecureStore`	LazySingleton	نسخة واحدة، تقدر تستنى
`Connectivity`	LazySingleton	نسخة واحدة، تقدر تستنى
`NetworkInfo`	LazySingleton	نسخة واحدة، تقدر تستنى
`SharedPreferences`	Singleton	async `getInstance()`
`LocaleCubit`	Factory	`BlocProvider` بيقفلها
`TokenStore`	Singleton	لازم تخلص قبل الـ router `restore()`
`ApiClient`	LazySingleton	سلسلة interceptors واحدة للتطبيق كله
`AuthRemoteDataSource`	LazySingleton	مفيش حالة
`AuthRepository`	LazySingleton	مفيش حالة، ومسجّل بالعقد
`LoginUseCase` وإخواتها	Factory	خفيفة ومفيهاش حالة
`AuthCubit`	Factory	`BlocProvider` بيقفلها